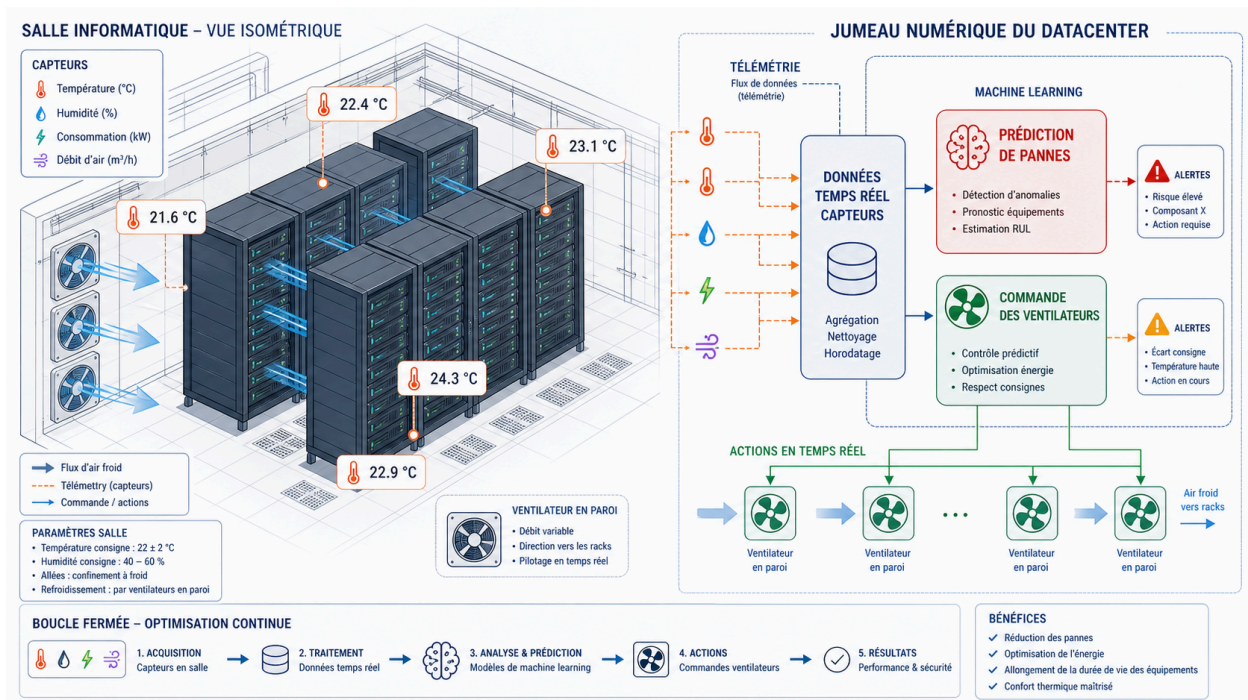
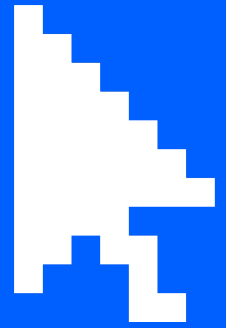


Juste des ventilateurs

Analyse de données temps réel émises par des matériels IoT



Quand les choses se réchauffent et que les capteurs s'agitent, dans un système qui contrôle juste des ventilateurs, notre IA est activée pour refroidir les choses et faire baisser la pression.

Introduction du sujet

Dans la plupart des datacenters modernes, la question n'est plus seulement de "faire tourner les serveurs", mais de **maintenir les machines dans une zone de fonctionnement sûre tout en contenant la facture énergétique**. Au quotidien, des



équipes d'ingénieurs data et d'experts IA doivent jongler entre alertes de température, tickets d'incident et objectifs de réduction de consommation électrique, avec des infrastructures déjà en production et peu de marge pour expérimenter.

Dans ce projet, vous travaillez sur un environnement contrôlé qui reproduit un parc de machines et ses contraintes thermiques. Le simulateur open source [jumeaux-chauds](#) joue le rôle de jumeau numérique d'un datacenter : télémétrie temps réel via MQTT, API de pilotage des ventilateurs, mécanismes d'arrêt automatique pour protéger les machines, scénarios de charge et de stress variés. Vous n'avez pas à gérer la couche matérielle ou réseau, mais à exploiter ce jumeau comme le feraient des ingénieurs en production.

Votre mission consiste à **concevoir un service de supervision et de contrôle thermique piloté par la donnée et le machine learning**. Concrètement, il s'agit d'**anticiper** les situations à risque (dégradations, arrêts thermiques) et de **piloter** intelligemment les ventilateurs pour maintenir les machines dans une zone sûre tout en limitant l'énergie dépensée. L'objectif est d'insérer des modèles de *machine learning* dans une boucle de décision, avec des métriques convaincantes.

Vous livrerez au final deux briques principales : **un modèle de prédiction de pannes**, et un **contrôleur ou une politique de régulation des ventilateurs**, intégrés dans un service de supervision et évalués sur des scénarios réalistes. Vos résultats devront permettre de répondre à des questions concrètes: "réduisons-nous vraiment les incidents ?", "combien de shutdowns évités ?", "quel impact sur la consommation énergétique ?".



Mise en situation professionnelle

Contexte : Une infrastructure multi-machines

Vous faites partie d'une petite équipe de 2 à 4 ingénieurs (data, IA, ML/MLOps) en charge de la **fiabilité thermique d'un datacenter** de taille moyenne. L'infrastructure est déjà équipée de capteurs de température et de ventilateurs contrôlables à distance, et expose des **flux de télémétrie** et des APIs, mais le pilotage reste largement fondé sur des règles statiques (seuils fixes, modes ventilateur prédéfinis). Les opérateurs constatent régulièrement des **épisodes de surchauffe, des passages en mode dégradé et quelques arrêts thermiques de sécurité**, en particulier lors des pics de charge.

Pour pouvoir **expérimenter sans risquer de perturber la production**, l'équipe s'appuie sur un **système de jumeau numérique** du parc de machines. Ce système simule des clusters de serveurs, des capteurs IoT, des états (on, degraded, off), une protection thermique automatique, ainsi qu'un **scénario de stress** reproduisant des situations réelles : montée rapide de charge, dérive des capteurs, bruit de mesure, et pannes injectées. Le jumeau expose un **broker MQTT** pour la télémétrie, des endpoints REST pour le pilotage des ventilateurs et une **stack d'observabilité** (TimescaleDB, Grafana) proche de ce qu'on trouve en production.

Votre mission : Un service de supervision intelligent

Dans ce contexte, votre équipe doit proposer une stratégie crédible pour améliorer la robustesse thermique et la consommation énergétique, en s'appuyant sur des **modèles de machine learning intégrés au flux temps réel**. Votre service de supervision intelligent devra :



- **Lire et historiser la télémetrie** en continu pour construire un jeu de données exploitable en apprentissage.
- **Entraîner un modèle capable de prédire**, avec un horizon utile, les situations menant à une panne ou à un mode dégradé (surchauffe, arrêt de sécurité, dépassement de seuil critique).
- **Définir un contrôleur** ou une politique d'action pour ajuster les consignes des ventilateurs (modes ou vitesses), en cherchant un compromis entre sécurité thermique et sobriété énergétique.
- S'insérer dans une **boucle opérationnelle** réaliste : lire l'état, évaluer le risque, décider une action, envoyer les ordres de ventilation via API, puis observer et mesurer l'impact sur la température et sur la consommation.

Le simulateur vous sert de terrain d'essai pour **comparer plusieurs stratégies** : comportement natif du système, règles à seuils simples, contrôleur avancé piloté par modèle. À l'issue du projet, vous devez être capables de documenter, chiffres à l'appui, ce qu'une équipe d'ingénieurs data/IA pourrait réellement gagner en déployant ce type de supervision dans un datacenter : moins d'incidents thermiques, moins de pics de température, et une consommation d'énergie mieux maîtrisée pour le refroidissement.

Enjeux clés

- Sécurité thermique : ne jamais tolérer un shutdown par dépassement critère de température.
- Efficacité énergétique : réduire la consommation sans sacrifier la sûreté. Un bon contrôleur garde les machines assez froides avec le minimum d'énergie compatible avec la sûreté, pas à tout prix.
- Reproductibilité : tous les résultats doivent être reproductibles et documentés pour la validation en contexte réel.



Travaux à réaliser

Vous devez livrer un projet reproductible “juste des ventilateurs”, exécutable côte à côte avec jumeaux-chauds, idéalement sous forme d'un microservice Docker. Voici la roadmap et les phases clés.

Phase 1 : Prise en main et compréhension

- Cloner et lancer jumeaux-chauds avec le scénario stress.
- Vérifier et explorer la télémétrie MQTT (topics, payloads, structure).
- Identifier les endpoints REST de contrôle des ventilateurs (fan_mode, fan_speed).
- Comprendre les états (on, degraded, off) et la logique de protection thermique d'arrêt automatique.

Phase 2 : Ingestion et stockage des données

- Développer un subscriber MQTT fiable pour collecter les télémétries en temps réel.
- Parser et normaliser les payloads en un schéma unifié (timestamp, cluster, machine, load, température, RPM, puissance, événements).
- Stocker dans TimescaleDB pour les requêtes temps réel et agrégations, ou si vous le préférez en Parquet/CSV versionné pour les datasets d'apprentissage reproductibles.
- Construire des exports train/validation/test par épisode ou seed.

Phase 3 : Feature engineering

- Construire des features temps réel : températures actuelles et dérivées (5s, 15s, 30s), marge au shutdown, charge et rolling means, RPM et variance.



- Ajouter des indicateurs contextuels : durée passée en zone chaude, indicateurs de défaut/degraded récents, historique des changements de consigne.
- Créer des features énergétiques : puissance instantanée, consommation cumulée, coût estimé avec PUE.

Phase 4 : Modèle d'anticipation de pannes

Objectif : Prédire, avec assez de temps d'avance, un incident futur (passage en degraded, shutdown thermique, dépassement critique).

- Définir précisément les labels : failure_60s = 1 si degraded/off dans les 60s suivantes, hot_30s = 1 si température dépasse un seuil critique.
- Baseline obligatoire : seuils heuristiques (ex. si température > T_warn depuis N secondes, alors panne probable).
- Modèles recommandés : Logistic Regression, Random Forest, ou Gradient Boosting (XGBoost/LightGBM si maîtrisé).
- Évaluation : Precision, Recall, F1, PR-AUC, et surtout temps moyen d'anticipation avant incident. Le Recall sur cas dangereux doit être élevé.
- Validation : Séparation train/test par épisodes, test sur plusieurs seeds, analyse critique des faux négatifs.

Phase 5 : Contrôleur de régulation des ventilateurs

Objectif : Choisir une consigne de ventilation pour maintenir la sécurité thermique et réduire la consommation.

- Baselines obligatoires : ventilateur fixe, contrôle à seuils (ex. if $T > T_l$ then RPM=high), proportionnel ou PID simple.
- Actions discrétisées : RPM $\in \{0, 1500, 2500, 3500, 4500\}$ par exemple, pour rester faisable en classification ou bandit simple.



- Stratégies recommandées : apprentissage supervisé d'une politique (classifier qui reproduit une 'meilleure' action offline) ou contrôle à score (combinaison pondérée de sécurité + énergie).
- Exemple de coût multi-objectif : $J(t) = \alpha \cdot \text{risk}(t) + \beta \cdot \text{heat}(t) + \gamma \cdot \text{energy}(t) + \delta \cdot |\Delta \text{RPM}|$ pour équilibrer sécurité et sobriété.
- Option avancée pour groupes experts : contextual bandit ou RL léger (pas requis pour une très bonne note).

Phase 6 : Boucle fermée et évaluation

- Brancher le contrôleur en temps réel, passer les machines en mode manual pour les soumettre à votre pilotage.
- Garder l'arrêt thermique automatique actif (c'est l'événement d'échec à éviter).
- Enregistrer toutes les décisions prises et les résultats observés.
- Comparer sur un même scénario et durée : baseline auto native vs baseline seuils/PID externe vs votre modèle.
- Tester la robustesse : variation de charge, bruit capteur, drift plus rapide, pannes fréquentes, généralisation à d'autres scénarios.

Livrables minimaux

- Un dépôt juste-ventilateurs avec code source, README, instructions d'exécution et dépendances.
- Module d'ingestion MQTT et stockage unifié (TimescaleDB ou fichiers).
- Dataset d'apprentissage versionné ou reproductible (avec seed et paramètres).
- Modèle de prédiction de pannes entraîné et sauvegardé.
- Contrôleur de régulation des ventilateurs (classifieur ou politique à score).
- Protocole d'évaluation clair et reproductible (splits, métriques, résultats).



- Rapport synthétique comparant plusieurs approches avec graphiques et conclusions.

Structure de dépôt recommandée

juste-ventilateurs/

```
|— ingest/                                # Collecte MQTT et normalisation
|— features/                             # Ingénierie des features
|— models/
|   |— failure_prediction/               # Modèle d'anticipation
|   |— fan_control/                     # Contrôleur/politique
|— evaluation/                           # Protocole et métriques
|— notebooks/                            # Analyses et explorations
|— data/                                 # Datasets versionés
|— docker-compose.yml                    # Branchement au simulateur
|— README.md
|— requirements.txt
|— setup.py
```

Compétences visées

Compétences data & temps réel

- Collecte et prétraitement de données de télémétrie en temps réel (ingestion, nettoyage, normalisation, construction de datasets reproductibles).
- Mise en place de pipelines ETL et de flux de données temps réel pour l'analyse et la détection d'anomalies.



- Mise en place de solutions de stockage adaptées (data lakes, data warehouses) et gestion de la gouvernance des données (qualité, traçabilité, conformité).

Compétences en analyse et machine learning

- Analyse descriptive des télémétries, identification de patterns et détection d'anomalies exploratoires.
- Développement, entraînement, validation et optimisation de modèles supervisés pour la classification, la prédiction et la détection d'anomalies en temps réel.
- Visualisation des tendances, des comportements anormaux et des résultats des modèles.

Compétences en ingénierie de données et MLOps

- Conception et gestion de pipelines de données reproductibles (architecture ETL, versioning des datasets et des modèles, CI/CD).
- Déploiement de systèmes d'analyse temps réel et de solutions ML en production (microservices, APIs, orchestrations de flux).
- Mise en place de mécanismes de monitoring, d'observation des métriques et de feedback loops pour améliorer les modèles et les pipelines.

Compétences en architecture et intégration

- Conception d'architectures data pour l'analyse temps réel, du collecteur aux couches de stockage et de service.
- Gestion des flux de données via des APIs et intégration avec des briques applicatives ou IoT existantes.
- Prise en compte des contraintes de performance, de scalabilité et de fiabilité dans l'architecture globale.



Compétences transverses

- Gestion de projet et collaboration au sein d'une équipe pluridisciplinaire (planification, répartition des tâches, communication).
- Documentation, rédaction de rapports, préparation de visualisations et de supports de présentation pour valoriser les résultats.
- Présentation et démonstration des capacités du système et du pipeline de données auprès de publics techniques et non techniques.

Rendu

Les livrables attendus regroupent le code, les données, les modèles et un rapport d'analyse.

Code et dépôt

- Dépôt public (GitHub, GitLab) avec historique clair et commits explicatifs.
- README complet : contexte, installation, exécution, dépendances, résultats attendus.
- Docker ou docker-compose pour reproductibilité (pré-requis recommandé).

Modèles et datasets

- Modèles entraînés sauvegardés (pickle, joblib, ONNX ou format framework).
- Datasets train/validation/test en Parquet ou CSV, avec description du schéma.
- Hyperparamètres et logs d'entraînement documentés.

Rapport d'analyse

- Introduction et motivation technique (5-10 pages).



- Description de l'architecture et du pipeline de données.
- Résultats des modèles : matrices de confusion, courbes PR/ROC, benchmarks comparatifs.
- Métriques de sécurité et énergie : taux de shutdown, consommation, gain par rapport aux baselines.
- Analyse de robustesse et limitations.
- Perspectives et travaux futurs.

Base de connaissances

Les ressources ci-dessous vous aideront à monter en compétence sur les technologies et concepts clés du projet.

IoT et temps réel

- MQTT Guide : <https://mqtt.org/>
- Mosquitto MQTT Broker : <https://mosquitto.org/>
- Apache Kafka : <https://kafka.apache.org/>

Stockage et bases de données temps réel

- TimescaleDB (série temporelle) : <https://www.timescale.com/>
- DuckDB (requêtes locales rapides) : <https://duckdb.org/>
- Apache Parquet (format columnar) : <https://parquet.apache.org/>

Machine Learning et données temporelles

- scikit-learn (ML classique) : <https://scikit-learn.org/>
- XGBoost : <https://xgboost.readthedocs.io/>
- LightGBM : <https://lightgbm.readthedocs.io/>
- Pandas et series temporelles :
https://pandas.pydata.org/docs/user_guide/timeseries.html



Déploiement et observabilité

- Docker et containerisation : <https://www.docker.com/>
- Prometheus (monitoring) : <https://prometheus.io/>
- Grafana (dashboards) : <https://grafana.com/>

Projet jumeaux-chauds

GitHub repository : <https://github.com/tristanv/jumeaux-chauds>